

Practica 11: Fundamentos de Simulación

En esta práctica veremos los fundamentos para simular números aleatorios. Este es un campo de suma importancia en un sin fin de áreas de gran relevancia en la actualidad. A lo largo de la práctica deberán simular variables aleatorias las cuales seguirán diferentes leyes de distribución. Una buena forma de corroborar si los datos simulados verdaderamente se ajustan a la distribución deseada es por medio de un histograma.

Un histograma es una representación gráfica de datos agrupados mediante intervalos. Gracias a él puedes hacerte rápidamente una idea de la distribución de los datos o muestra. Un histograma es un conjunto de barras rectangulares verticales que su altura es proporcional a las frecuencias absolutas de cada uno de los intervalos.

Por eso mismo es importante saber como hacer un histograma dado un vector de datos. Para nuestra fortuna python cuenta con la paquete `matplotlib` el cual nos brinda las funciones necesarias para generar histogramas con gran facilidad. A continuación veremos un ejemplo de como usar esa función. Supongamos que tenemos la siguiente lista con datos generados por medio varios experimentos aleatorios:

```
Valores = [-1.55343781, -1.0928067, -0.14113104, 1.18801628, -  
0.56328972, -0.48574817, -0.10387042,  
-0.77310551, -0.75269235, -2.06276255, 0.33171956,  
0.18060089, -0.28371874, 0.18654273,  
0.01886643, -1.03615981, -1.20024984, -0.32038366,  
1.14537454, -0.05386294, -0.26336796,  
-0.65782501, 1.26941747, -0.56547108, 0.46742205,  
0.5484787, -0.0173109, 0.7412696,  
-0.16532867, -2.04408291, 0.71707934, -0.01179361,  
0.08356211, 1.32462912, 2.08337469,  
0.05412171, 1.1096192, 0.47054017, -1.55714503,  
0.89552254, 1.45788242, -0.53235342,  
-0.7484324, 1.08845687, 0.91340503, 2.23171891,  
1.20269166, 0.72453522, -0.43762806,  
0.06844963, 1.89060303, 0.69869295, -1.32415541, -  
0.95434851, 0.01556138, -0.69104306,  
0.31521161, 0.76072805, 0.4950435, -0.51300597,  
0.91674249, -1.56242603, 0.19326191,  
0.26072427, -0.73257999, -1.02799671, -0.47870884,  
0.70264922, -1.84317905, 0.79133609,  
-0.6421007, -0.25286489, -0.32871562, -0.51101343,  
0.65977296, -0.11277094, 0.63768946,  
-0.59555345, 0.53923943, -0.04820176, -0.25946007,  
0.393552, -0.02432366, 1.45970927,  
-1.09002175, 0.31458349, 1.20102083, 0.87005601, -  
0.06621691, -0.16124722, -1.29900607,  
1.22508168, 1.17870723, -0.46641485, -1.77726474, -
```

```
0.93051079, -1.41144336, -0.9303475 ,  
0.32221209, -0.56898838]
```

Podemos generar el histograma de estos datos por medio de la función `hist` de `matplotlib.pyplot`. A continuación un ejemplo de como usar esta función y algunos parámetros que puede modificar para cambiar su apariencia.

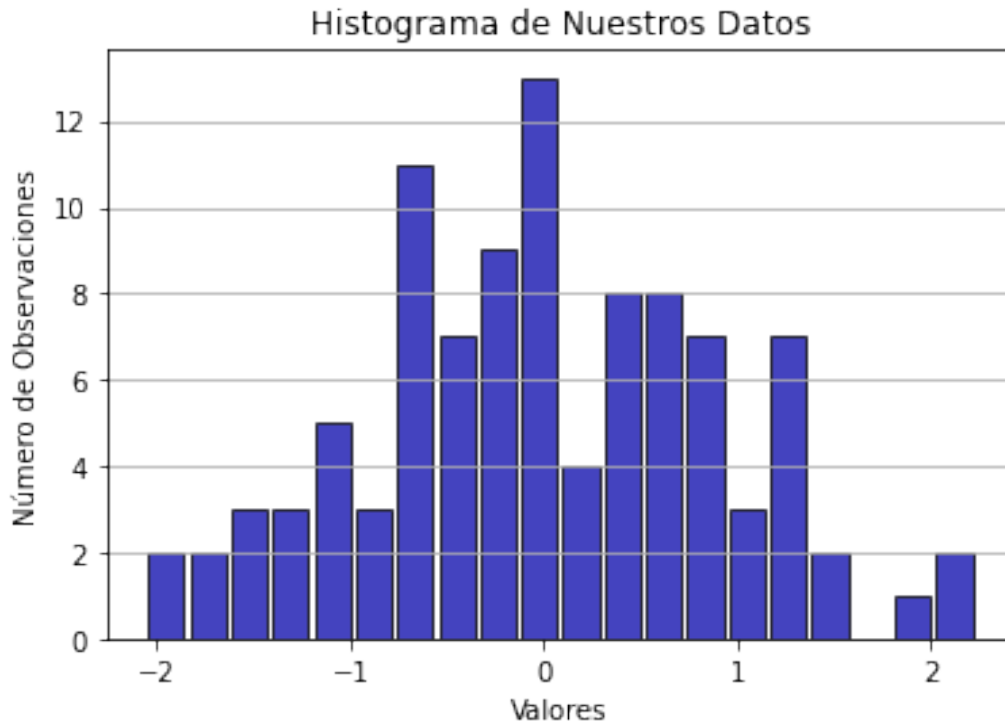
Para más información pueden visitar

https://matplotlib.org/3.3.3/api/_as_gen/matplotlib.pyplot.hist.html

- **bins** - Número de barras en el histograma.
- **color** - Color de las barras en histograma.
- **alpha** - Transparencia de las barras.
- **rwidth** - el ancho de las barras.
- **edgecolor** - Color de los bordes de las barras.

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
plt.hist(Valores, bins = 20, color= '#0504aa', alpha=0.75,  
rwidth=0.85, edgecolor='black')  
plt.xlabel('Valores')  
plt.ylabel('Número de Observaciones')  
plt.title('Histograma de Nuestros Datos')  
plt.grid(axis = 'y')  
plt.show()
```



Como pueden observar del histograma anterior, las alturas de las barras corresponden al total de datos observados de cada uno de los intervalos que abarcan las barras. Sin embargo, en vez de tomar el total de datos observados también podemos tomar la densidad de los datos. Para esto simplemente se toma la altura de las barras como el número de datos observados entre el total de datos.

Para poder obtener el histograma de las densidades simplemente debemos agregar el parámetro `density = True` dentro de la función `hist`. Modifica el código anterior para obtener el histograma de la densidad y observa el cambio de los valores en el eje vertical.

Ejercicio 1:

Uno de los resultados más importantes en la teoría de la probabilidad es **la ley débil de los grandes números**.

Lo que nos dice la ley débil de los grandes números es que si tenemos una sucesión de variables aleatorias $X_1, X_2, \dots, X_n$ independientes e idénticamente distribuidas, entonces su promedio

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n X_i \rightarrow E[X_i]$$

converge en probabilidad a la esperanza de las variables aleatorias cuando n tiende a infinito

En este ejercicio deberán programar una función que nos ayude a visualizar este resultado.

Su función deberá recibir:

- Una lista de valores
- Calcular los promedios parciales de estos datos

$$S_k = \frac{1}{k} \sum_{i=1}^k x_i, k=1, \dots, N$$

- Graficar los valores calculados de los promedios parciales.

Prueben su función generando una lista de 200 valores aleatorios provenientes de una distribución *binomial*(7, 0.8), usando el siguiente código:

```
np.random.binomial(n, p, size)
```

A tu gráfica anterior agregale la línea constante $y = np = 7 \cdot 0.8$

Ejercicio 2:

Como ya saben, se puede usar una variable aleatoria con distribución Uniforme(0,1) para simular nuevas variables aleatorias que distribuyan conforme a diferentes leyes. La única función que podrán usar será `uniform()` del módulo `numpy.random` para generar las variables aleatorias con distribución Uniforme(0,1).

Variable Aleatoria Bernoulli

Programa una función que simule una variable aleatoria con distribución Bernoulli con probabilidad de éxito p .

$$f(x) = \begin{cases} 1 - p & \text{si } x = 0 \\ p & \text{si } x = 1 \\ 0 & \text{en otro caso} \end{cases}$$

Variable Aleatoria Geométrica

Programa una función que simule una variable aleatoria con distribución Geométrica con probabilidad de éxito p .

Pista: Recuerda que la variable aleatoria geométrica se puede interpretar como el número de ensayos Bernoulli necesarios antes de obtener un éxito.

Variable Aleatoria Exponencial

Usando el teorema de la función inversa, programa una función que simule una variable aleatoria con distribución exponencial con parámetro λ . Recuerden que la función de distribución acumulada de la exponencial con parámetro λ es:

$$F(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 - e^{-\lambda x} & \text{si } x \geq 0 \end{cases}$$

Variable Aleatoria Pareto

Usando el teorema de la función inversa, programa una función que simule una variable aleatoria con distribución Pareto con parámetro de escala $c > 0$ y parámetro de forma $\alpha > 0$. Recuerden que la función de distribución acumulada de la exponencial con parámetro λ es:

$$F(x) = \begin{cases} 0 & \text{si } x < c \\ 1 - \left(\frac{c}{x}\right)^\alpha & \text{si } x \geq c \end{cases}$$

Ejercicio 3:

Ahora, como todo buen probabilista, nos meteremos un poco en el mundo de los juegos de azar. En este ejercicio deberán simular una variable aleatoria que simule un dado justo y una que simule un dado cargado.

Dado Justo:

Deberán simular un dado justo que tenga k caras; es decir que puede salir cada cara con probabilidad $1/k$. Programen una función que reciba el número de caras que tendrá el dado y regrese alguna de ellas con igual probabilidad.

Pista: Recuerden la función de distribución de la uniforme discreta... tal vez eso les ayude.

Dado Cargado:

Ahora, queremos modelar un dado cargado con k caras; es decir que no todas las caras tienen la misma probabilidad de salir. Para poder simular este dado su función deberá recibir un vector de probabilidades $(p_1, p_2, \dots, p_k)$ donde p_i es la probabilidad de obtener la i -ésima cara del dado. Recuerden que se debe cumplir que $p_1 + p_2 + \dots + p_k = 1$.

Pista: Su función debe ser muy similar a la del dado justo pero ahora en lugar de aparecer las probabilidades $1/k$ deberán aparecer las p_i